

“Event and Room Occupancy Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KAMTAM PRIYANKA 2520090001

JALLA GEYA REDDY 2520090123

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Event and Room Occupancy Management System”** is a record of bonafide work of **KAMTAM PRIYANKA – 2520090001, JALLA GEYA REDDY - 2520090123** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KAMTAM PRIYANKA – 2520090001

JALLA GEYA REDDY – 2520090123

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Event and Room Occupancy Management System”** is a bonafide work done and submitted by **KAMTAM PRIYANKA – 2520090001, JALLA GEYA REDDY – 2520090123** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. R. Sateesh Kumar** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

KAMTAM PRIYANKA – 2520090001

JALLA GEYA REDDY – 2520090123

Abstract

The Event and Room Occupancy Management System is a web-based application designed to help students and college administrators efficiently manage room availability, event scheduling, and participant registrations within a college campus. Students often face difficulty finding available rooms during college hours, while events such as hackathons, workshops, seminars, and competitions may experience unexpected increases in participants due to spot registrations. These situations can lead to room allocation conflicts, overcrowding, and inefficient utilization of available spaces.

The system provides a user-friendly web interface through which students can view available rooms, check room capacity and schedules, register for events, and access relevant event information. Administrators are able to create and manage events, allocate rooms, monitor registrations, and track occupancy. The system detects room scheduling conflicts and capacity issues, allowing administrators to make better decisions regarding room allocation.

In its final implementation, the application uses a React.js frontend built with Vite, and a backend divided into independent Node.js and Express.js microservices: a User Service, an Admin Service, and an Event Service. All frontend requests pass through a single API Gateway, which routes each request to the appropriate service and forwards the JWT Authorization header. Data is stored in MySQL, with a separate logical database for each service (`user_service_db`, `admin_service_db`, and `event_service_db`), so that each service owns its own data. Authentication is based on JSON Web Tokens with bcrypt-hashed passwords and role-based access for students and administrators. The Event Service manages events, rooms, room availability, room assignments, registrations, bookings, and occupancy, and applies a single overlap rule for room conflict detection together with an effective-occupancy calculation (actual occupancy plus registered participants, compared against expected participants).

The system was originally built as a single backend and was later restructured into the service-based architecture described in this report. It was built and tested in incremental phases against a running MySQL database. The result is a centralized platform that improves visibility of room availability, reduces scheduling conflicts, and supports the management of events and rooms on a campus.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		
	Abstract		
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend (Microservices) 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 API Gateway 4.3 User Service 4.4 Admin Service 4.5 Event Service 4.6 Database Separation 4.7 Request Flow 4.8 Deployment Architecture	10
5	Design	5.1 Database Design Principles 5.2 Database Distribution Across Services 5.3 ER Diagram / Database Schema 5.4 Design Decisions and Assumptions	13
6	Implementation	6.1 Microservice-based Backend Implementation 6.2 User Service Implementation 6.3 Admin Service Implementation 6.4 Event Service Implementation 6.5 API Gateway Implementation 6.6 Frontend API Integration 6.7 Authentication and Authorization 6.8 Inter-service Request Flow	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	19
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	21
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	23
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	24

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	25
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	26
13	References	Documentation, tools, and resources used	27
14	Appendices	Appendix A – Sample Code Snippets Appendix B – Installation / Setup Instructions Appendix C – Default Login Credentials Appendix D – API Reference Summary Appendix E – Screenshots	28

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Colleges routinely run a large number of events — workshops, seminars, hackathons, club meetings, and competitions — that all compete for the same limited pool of rooms. In most departments, room booking is still coordinated informally: through phone calls, WhatsApp messages, a shared spreadsheet, or a notice pinned outside the administration office. This works reasonably well when activity is low, but breaks down quickly during periods of high demand, such as a hackathon that unexpectedly attracts far more participants than a room can seat, or two departments independently booking the same seminar hall for overlapping time slots.

The Event and Room Occupancy Management System was built to replace this informal coordination with a single, database-backed source of truth. Every room, every event, and every registration is stored in a relational database, and the system itself — not a human cross-checking a spreadsheet — is responsible for detecting when a room is already booked, when a room is too small for an event, or when a student tries to register twice for the same event. The project was first implemented with a single backend and was subsequently restructured into an API Gateway and three backend microservices, as described in Chapters 4 to 6.

1.2 Problem Statement

Given a college campus with a fixed number of rooms of varying capacity and type, and a continuous stream of events being scheduled by different organizers, the system must answer three questions reliably and in real time: which rooms are free for a given date and time, whether a specific room can be safely assigned to a specific event, and whether a specific student can be allowed to register for a specific event. Answering these questions manually does not scale, is error-prone, and gives students no visibility into room availability until they physically check.

1.3 Scope of the Project

The system covers the full lifecycle of a campus event from the room's perspective: adding and maintaining a directory of rooms (with capacity, type, equipment, and status); creating and editing events; assigning a room to an event only after checking its status, capacity, and existing schedule; students registering for events subject to duplicate and capacity checks; an audit trail of booking actions; and an occupancy-tracking mechanism that records actual attendance and combines it with registered participants. Payment processing, multi-campus support, and a native mobile application were treated as out of scope for this submission.

1.4 Objectives

- Provide real-time room availability for any requested date and time window.
- Prevent a room from ever being double-booked, using a single, consistently-applied overlap rule.
- Enforce registration rules: no duplicate registrations, and no registrations beyond a room's capacity once one is assigned.
- Give administrators a single dashboard to manage rooms, events, bookings, registrations, and occupancy.
- Automatically calculate an effective occupancy percentage and status for events, based on actual occupancy, registered participants, and expected participants.

- Secure the system with JWT-based, role-based access, so student accounts can never reach admin-only functionality.
- Organize the backend as independent services (User, Admin, and Event) behind a single API Gateway, with each service owning its own database.

1.5 Importance of the Application

By centralizing room and event data, the system removes the guesswork from room allocation. Administrators gain a live view of which rooms are occupied at any given moment and which upcoming events have a capacity mismatch, letting them intervene before an event happens rather than discovering a problem on the day. Students gain the ability to check room availability and register for events themselves, without needing to contact an administrator directly.

1.6 Target Users / Audience

The system serves two roles. Students use it to browse and search events, view event details, register for events, track their own registrations, and check room availability. Administrators use it to manage the room inventory, create and edit events, assign rooms to events, monitor all bookings and registrations across the campus, and track occupancy.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has modest hardware requirements, since the Node.js services and the MySQL server are lightweight relative to the small dataset a single campus generates. In the final implementation, the entire stack runs on one computer as several independent processes: the frontend development server, the API Gateway, three backend services, and one MySQL server. A development or demonstration machine with a dual-core processor, 4 GB of RAM, and roughly 500 MB of free disk space (for Node modules of each project, MySQL data files, and source code) is sufficient.

Table 2.1: Hardware requirements

Component	Minimum Requirement
Processor	Dual-core processor
Memory (RAM)	4 GB
Free disk space	Approximately 500 MB
Network	Local machine (localhost); no external network required

2.2 Software Requirements

Table 2.2: Software requirements

Software	Requirement	Purpose
Node.js with npm	Version 18 or later	Runs the frontend tooling, API Gateway, and all three services
MySQL	Version 8.0 or later	Hosts the three logical service databases
Web browser	Chrome, Edge, or Firefox	Access to the React application
Git	Any recent version	Version control
MySQL Workbench, Visual Studio Code, Postman	Optional but recommended	Database inspection, development, and API testing

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) was used for the frontend; Node.js with the Express.js framework for the API Gateway and the backend services; MySQL as the relational database, accessed through the mysql2 driver; Postman for API testing and documentation; and Git with GitHub for source control. VS Code was the primary development environment, and MySQL Workbench was used for inspecting and verifying the database schemas and data directly.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React.js and bundled using Vite. React Router handles client-side navigation between the public (login/register), student, and admin areas, and Axios is used as the single HTTP client for every API call, wrapped in one central service module per resource (rooms, events, registrations, bookings, occupancy). All API calls use the base path /api and are sent to the API Gateway, not to the individual services. Styling is plain CSS with a small set of reusable design tokens (colors, spacing, card/table/badge styles) rather than a heavyweight UI framework, keeping the codebase easy to read and modify.

3.2 Backend (Microservices)

The backend is built with Node.js and Express.js and is divided into four separately running Express applications: an API Gateway, a User Service, an Admin Service, and an Event Service. The frontend sends every request to the API Gateway, which forwards it, together with the JWT Authorization header, to the service responsible for that request. Each service is responsible for its own routes, controllers, models, and database connection, which is made with the mysql2 driver. Authentication uses JSON Web Tokens, and passwords are hashed with bcrypt.

The backend was separated into services for the following reasons: (i) each service has one clearly defined responsibility — student accounts, administrator accounts, or events and rooms — so a change in one area does not require modifying unrelated code; (ii) each service owns its own database, so data ownership is explicit; (iii) services can be started, stopped, and debugged independently; and (iv) the frontend depends on a single gateway address instead of three service addresses. The project is a local, course-scale implementation; it does not include containerization, service discovery, or other production infrastructure.

Table 3.1: Technology stack

Layer	Technology	Role
Frontend	React.js, Vite, React Router, Axios	User interface for students and administrators
API Gateway	Node.js, Express.js	Single entry point; routes requests and forwards the Authorization header
User Service	Node.js, Express.js, mysql2, bcrypt, jsonwebtoken	Student registration, login, profile, authentication
Admin Service	Node.js, Express.js, mysql2, bcrypt, jsonwebtoken	Admin login, authentication, profile, administrator operations
Event Service	Node.js, Express.js, mysql2, jsonwebtoken	Events, rooms, availability, assignments, registrations, bookings, occupancy
Database	MySQL 8	user_service_db, admin_service_db, event_service_db
Testing	Postman, curl, MySQL client, browser	API and end-to-end verification

3.3 Database

MySQL 8 stores all persistent data. Instead of one database holding every table, the data is split into three logical databases, one per service: user_service_db, admin_service_db, and event_service_db. The Event Service database holds the event-related tables (events, rooms,

registrations, room_schedules, bookings, and occupancy_records). Within each database, referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (e.g. one registration per student per event), and data validity with CHECK constraints (e.g. no negative capacity, start time before end time). Chapter 5 describes the distribution in detail.

3.4 Version Control

Git was used throughout development for version control, with the project hosted on GitHub. The client, the API Gateway, and the three services are kept as separate project folders.

3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt (10 salt rounds) before being stored — plain-text passwords are never written to the database. Login sessions are stateless JSON Web Tokens (JWT), signed with a secret that is kept in each service's environment file and is not stored in the source code. Services that verify tokens use the same signing secret. Express middleware verifies the token on every protected route and enforces role-based access, ensuring student accounts cannot reach admin-only endpoints even if a request is crafted by hand

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The final system follows a microservice-based architecture. The React client communicates only with the API Gateway. The gateway routes each request to the User Service, the Admin Service, or the Event Service, and each service reads and writes its own MySQL database. Figure 4.1 shows the components, their ports, and the direction of requests. This architecture replaces the three-tier design (React frontend, single Express API, single MySQL database) of the original implementation.

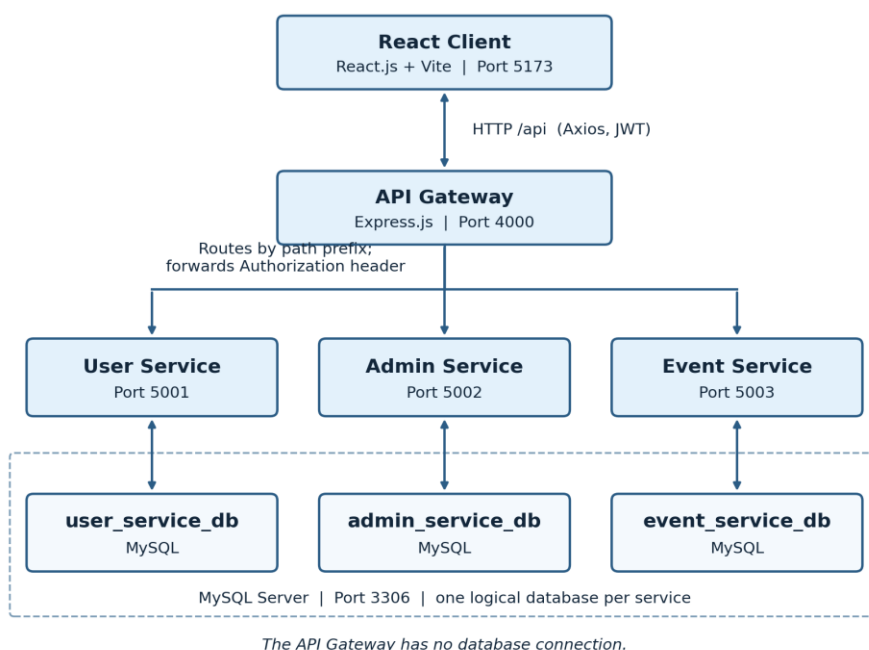


Figure 4.1: Final microservice architecture of the Event and Room Occupancy Management System

4.2 API Gateway

The API Gateway (port 4000) is the single entry point for frontend API requests. It does not connect to any database and contains no business logic. For each incoming request it selects the target service from the request path, forwards the request to that service, forwards the Authorization (JWT) header unchanged, and returns the service's response to the client. The gateway also provides a health endpoint that confirms it is running.

4.3 User Service

The User Service (port 5001) handles student registration, student login, the student profile, user authentication, and other user-related operations. It owns `user_service_db`. Registration through this service always creates a student account.

4.4 Admin Service

The Admin Service (port 5002) handles administrator login, administrator authentication, the administrator profile, and administrator-related operations. It owns `admin_service_db`. There is no public route for creating an administrator account.

4.5 Event Service

The Event Service (port 5003) handles events, rooms, room availability, room assignments, registrations, bookings, occupancy, and event/room scheduling with conflict checking. It owns

event_service_db. It verifies the JWT forwarded by the gateway and applies role checks, so that admin-only operations (for example room management and room assignment) are rejected for student tokens.

4.6 Database Separation

Each microservice owns one logical database inside the MySQL server (port 3306) and maintains its internal relationships independently. Cross-service communication is performed through service APIs rather than cross-service foreign keys. Table 4.1 summarizes the responsibilities and data ownership of each component.

Table 4.1: Microservice responsibilities and data ownership

Component	Port	Responsibility	Database
API Gateway	4000	Single entry point; routing; Authorization header forwarding; health endpoint	None
User Service	5001	Student registration, login, profile, authentication	user_service_db
Admin Service	5002	Admin login, authentication, profile, administrator operations	admin_service_db
Event Service	5003	Events, rooms, availability, assignments, registrations, bookings, occupancy, conflict checking	event_service_db

4.7 Request Flow

A typical request proceeds as follows. (1) The user acts in the React application, and an Axios call is made to the /api path with the JWT in the Authorization header. (2) The API Gateway on port 4000 receives the request and determines the target service from the path. (3) The gateway forwards the request, including the Authorization header, to the User, Admin, or Event Service. (4) The service verifies the token where the route is protected, applies its validation and business rules, and queries its own database through a mysql2 connection pool. (5) The response returns through the gateway to the browser. Login requests follow the same path but do not carry a token; the successful response contains the JWT that the frontend stores for later requests.

4.8 Deployment Architecture

For development and demonstration purposes, all components run as separate local processes on a single machine, started independently. Table 4.2 lists the components and ports. No containerization, CI/CD pipeline, or cloud deployment was configured for this submission; Chapter 11 outlines possible future extensions.

Table 4.2: Components and port numbers

Component	Technology	Port	Responsibility
React Client	React.js, Vite	5173	User interface; sends API requests to the gateway
API Gateway	Node.js, Express.js	4000	Routes requests to the services
User Service	Node.js, Express.js	5001	Student accounts and authentication
Admin Service	Node.js, Express.js	5002	Administrator accounts and authentication

Component	Technology	Port	Responsibility
Event Service	Node.js, Express.js	5003	Events, rooms, registrations, bookings, occupancy
MySQL Server	MySQL 8	3306	Hosts the three service databases

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was normalized to remove redundant data: each fact is stored in exactly one place, and every relationship between entities within a database is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database — not just in application code — using PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK constraints, so that invalid data (a negative capacity, an end time before a start time, a duplicate registration) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 Database Distribution Across Services

The original design used one database containing seven tables. In the final system, this data is distributed across three logical databases according to the owning service, as shown in Table 5.1.

Table 5.1: Database distribution

Service	Logical Database	Tables / Data	Description
User Service	user_service_db	users	Student account data with bcrypt-hashed passwords
Admin Service	admin_service_db	admins	Administrator account data with bcrypt-hashed passwords
Event Service	event_service_db	events, rooms, registrations, bookings, room_schedules, occupancy_records	Event, room, registration, booking, schedule, and occupancy data

5.3 ER Diagram / Database Schema

Figure 5.1 shows the database architecture of the final system. Each service database appears as a separate boundary. Relationships are drawn only inside the Event Service database, where the tables are linked by foreign keys; no foreign keys are drawn between the separate service databases. Each microservice owns its database/schema and maintains its internal relationships independently, and cross-service communication is performed through service APIs rather than cross-service foreign keys.

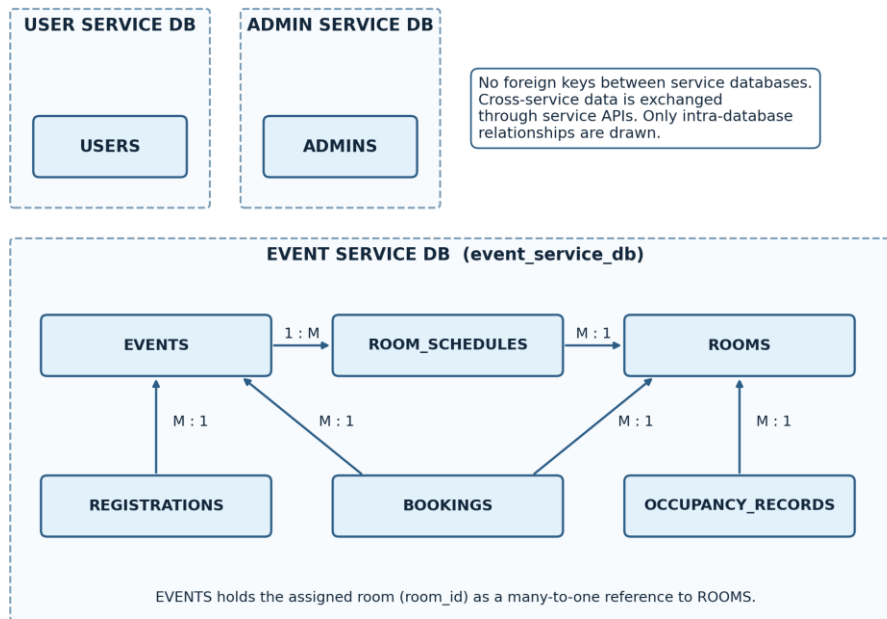


Figure 5.1: ER / database architecture showing service boundaries and Event Service relationships

USERS (User Service DB) stores each student account with a bcrypt-hashed password. ADMINS (Admin Service DB) stores each administrator account. In the Event Service DB, ROOMS holds the physical inventory of bookable spaces — name, number, building, capacity, type, equipment, and status. EVENTS holds each event's details and an optional assigned room. REGISTRATIONS links a student to an event they have signed up for, with a UNIQUE constraint preventing the same student registering twice for the same event. ROOM_SCHEDULES holds the authoritative time-block for every room-event pairing and is the single table every conflict check reads from. BOOKINGS records which administrator made a given room assignment and when, serving as an audit trail rather than a second source of truth for conflict-checking. OCCUPANCY_RECORDS stores expected, registered, and actual attendance figures per event, room, and date.

Because the users and admins live in other services, identifiers of students and administrators stored in Event Service tables (for example in registrations and bookings) are carried as values taken from the verified JWT, not as foreign keys into another database

5.4 Design Decisions and Assumptions

The following design decisions were made explicitly and are recorded here rather than left implicit in the code:

- **Data ownership by service:** each service owns exactly one logical database, and no service queries another service's database directly.
- **ROOM_SCHEDULES vs. BOOKINGS:** since both tables could plausibly record “room + time,” ROOM_SCHEDULES was designated the single source of truth that every availability check and every room-assignment reads from, while BOOKINGS is treated purely as an audit trail of who performed the assignment. Both are written together, atomically, inside one database transaction, so they can never drift out of sync.

- **Effective registration capacity:** the EVENTS table has no dedicated “maximum participants” field, so once a room is assigned to an event, that room's capacity is treated as the effective registration limit. Before a room is assigned, registration is left uncapped.
- **Effective occupancy:** occupancy is evaluated against the event's expected participants using actual occupancy plus registered participants, as described in Section 7.2.

CHAPTER 6: IMPLEMENTATION

6.1 Microservice-based Backend Implementation

The original single Express backend, which contained six resource modules (Authentication, Rooms, Events, Registrations, Bookings, and Occupancy), was restructured into an API Gateway and three services. Authentication was divided between the User Service (students) and the Admin Service (administrators), and the Rooms, Events, Registrations, Bookings, and Occupancy modules were grouped in the Event Service. Each service is an independent Express application with its own configuration file, its own port, and its own database connection pool. The services are started separately, and the frontend reaches all of them through the gateway.

Table 6.1: Mapping of original modules to final services

Original Module	Final Location
Authentication (students)	User Service
Authentication (administrators)	Admin Service
Rooms	Event Service
Events	Event Service
Registrations	Event Service
Bookings	Event Service
Occupancy	Event Service

6.2 User Service Implementation

The User Service exposes routes for student registration, login, and retrieval of the logged-in student's profile. On registration, the password is hashed with bcrypt and the account is stored with the student role in `user_service_db`. On login, the submitted password is compared with the stored hash and, if valid, a JWT carrying the user's ID, role, and name is returned. Registration always creates a student; no route creates an administrator.

6.3 Admin Service Implementation

The Admin Service exposes routes for administrator login and the administrator profile, using `admin_service_db`. Administrator accounts are not created through a public route; they exist because they were inserted into the database. A successful login returns a JWT carrying the administrator's identity and the admin role.

6.4 Event Service Implementation

The Event Service contains the project's main business logic and is organized into routes, controllers, and models. Its most involved piece is the room-assignment function, which runs checks in sequence — the room exists and is active, its capacity is sufficient for the event's expected participants, and it has no conflicting schedule for the requested time — and only if all checks pass does it atomically update the event's assigned room, write a new `room_schedules` row, and log a bookings row, inside a single database transaction so a failure partway through leaves no partial state behind.

The core conflict-detection rule used throughout the service is: a requested time slot conflicts with an existing schedule if $(\text{existing_start} < \text{requested_end}) \text{ AND } (\text{existing_end} > \text{requested_start})$.

This single rule is applied consistently by the room-availability search, the event room-assignment workflow, and the booking-creation endpoint, so a room can never end up double-booked no matter which entry point was used to book it.

6.5 API Gateway Implementation

The API Gateway is an Express application listening on port 4000. It contains a route table that associates path prefixes with the User, Admin, and Event Services. For a matching request it forwards the method, path, body, and Authorization header to the target service and returns the service's response. It exposes a health endpoint, and it has no database configuration.

6.6 Frontend API Integration

All HTTP calls go through one central Axios instance whose base path is /api. During development, Vite forwards /api requests to the API Gateway on port 4000, so the browser never needs to know the address of any individual service. The Axios instance automatically attaches the logged-in user's JWT to every outgoing request and automatically logs the user out if any response comes back with a 401 Unauthorized status (for example, after the token expires). Each resource has a matching frontend “service” file (roomService.js, eventService.js, and so on) so that no page ever hardcodes a URL or repeats the same fetch logic.

The frontend uses React's Context API for the two pieces of state that genuinely need to be shared across the whole application — the logged-in user (AuthContext) and toast notifications (ToastContext) — and otherwise keeps state local to the component that needs it. Shared UI elements (a Modal, a ConfirmDialog, status badges, and loading/empty/error placeholders) are each written once and reused across every admin and student page.

6.7 Authentication and Authorization

Student authentication follows the path Frontend → API Gateway → User Service, and administrator authentication follows the path Frontend → API Gateway → Admin Service. Each login returns a JWT. For later requests the frontend sends the token in the Authorization header; the gateway forwards this header unchanged, and the target service verifies it with the shared signing secret (which is kept in the environment configuration and is not reproduced in this report) and reads the role from the token.

Protected event, room, and administrator operations in the Event Service run a `protect` middleware that verifies the JWT, and admin-only routes additionally run an `adminOnly` middleware that checks the token's role claim, rejecting any student token with a 403 Forbidden response. Registration always creates a student account, so a user cannot register their way into administrator access.

6.8 Inter-service Request Flow

Table 6.2 describes how requests move between components for the main workflows. The services do not access one another's databases; the link between the services is the JWT issued by the User or Admin Service and verified by the Event Service.

Table 6.2: Request flow for the main workflows

Workflow	Request Path	Notes
Student login	Frontend → Gateway → User Service → user_service_db	JWT returned to the frontend

Workflow	Request Path	Notes
Admin login	Frontend → Gateway → Admin Service → admin_service_db	JWT returned to the frontend
Browse events / room availability	Frontend → Gateway → Event Service → event_service_db	Requires a valid token
Register for an event	Frontend → Gateway → Event Service → event_service_db	Student token; duplicate and capacity checks
Assign room to an event	Frontend → Gateway → Event Service → event_service_db	Admin token; status, capacity, and conflict checks in one transaction
Record occupancy	Frontend → Gateway → Event Service → event_service_db	Admin token; effective occupancy calculated

6.9 API Integration

Previously, the frontend called a single Express backend. Now it calls the API Gateway, which forwards each request to the appropriate microservice. Table 6.3 lists the main routes by service.

Table 6.3: Client requests, gateway routes, and target services

Client Request	Gateway Route	Target Service	Port
Student register / login / profile	/api/auth (register, login, me)	User Service	5001
Admin login / profile	/api/admin	Admin Service	5002
Rooms and room availability	/api/rooms	Event Service	5003
Events and room assignment	/api/events	Event Service	5003
Registrations	/api/registrations	Event Service	5003
Bookings	/api/bookings	Event Service	5003
Occupancy records	/api/occupancy	Event Service	5003
Gateway health	Health endpoint	API Gateway	4000

CHAPTER 7: FEATURES

7.1 List of Main Features

All features below are implemented through the microservice architecture: account features through the User and Admin Services, and all other features through the Event Service, each reached via the API Gateway.

Student-facing features:

- Register and log in with a student account
- Browse, search, and filter upcoming events by name and type
- View full event details, including assigned room and capacity
- Register for an event, with duplicate-registration and capacity limits enforced automatically
- View “My Registrations” with live status (Registered / Cancelled / Attended)
- Check room availability for any date and time window
- View a personal profile

Administrator-facing features:

- Log in with an administrator account and view a live dashboard (total rooms, rooms occupied right now, upcoming events, today's events, total registrations, and capacity warnings)
- Add, edit, and safely delete rooms (with a warning before shrinking a room's capacity below what upcoming events need)
- Add, edit, and delete events, and assign a room to an event, with room conflict detection and capacity monitoring
- View and manage all bookings, including cancellation
- View all student registrations, filterable by event or status
- Record and update occupancy after an event, with an automatically calculated effective occupancy percentage and status

7.2 How Each Feature Works

Room availability checking: given a date, a start time, an end time, and an optional minimum capacity, the system checks every room's status (excluding Maintenance/Inactive rooms), its capacity, and whether any existing room_schedules entry overlaps the requested window, returning two lists — rooms that are available, and rooms that are not, each with a plain-English reason such as “Already booked for ‘AI Workshop’ from 10:00 AM to 12:00 PM.” (Figure E.5)

Effective occupancy calculation: in the final implementation, occupancy is not simply actual occupancy divided by room capacity. The system computes:

$$\text{Effective Occupancy} = \text{Actual Occupancy} + \text{Registered Participants}$$

$$\text{Occupancy Percentage} = (\text{Effective Occupancy} \div \text{Expected Participants}) \times 100$$

If the percentage is 100 or more, the status is shown as “Over Capacity.” The percentage and status are displayed to the administrator as a coloured progress bar (Figure E.11). The remaining status labels from the original implementation (Normal, High, and Near Capacity) are retained as lower

bands.

Capacity-reduction warning: if an admin tries to reduce a room's capacity below what an upcoming event assigned to that room expects, the system does not silently allow it — it responds with the specific list of affected events and asks the admin to explicitly confirm before the change is applied (Figure E.9).

Room conflict detection: when a room is assigned to an event, the Event Service checks the room's status, capacity, and schedule. A request whose time window overlaps an existing schedule for that room is rejected with a message naming the clashing event; back-to-back events (one starting exactly when another ends) are allowed.

CHAPTER 8: TESTING

8.1 Postman API Testing

A Postman collection was built to exercise the REST endpoints. In the final system the requests are organized by service (User, Admin, Event) and also against the API Gateway. The Login requests carry a small test script that automatically saves the returned JWT into a collection variable, and requests such as Create Event, Assign Room, Register, and Create Occupancy Record are chained together using those variables, so a full workflow can be exercised without manually copying any ID or token between requests.

8.2 Integration Testing

The system was tested end-to-end against a real, running MySQL database at every stage of development — each backend module was verified with the actual database before the next was built on top of it, rather than being tested in isolation with mocked data. After the move to microservices, the same workflows were exercised through the API Gateway, so that routing, JWT forwarding, and each service's database access were tested together.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing, directly against each service and through the gateway
- curl (command line) — scripted regression testing
- MySQL command-line client and Workbench — direct verification of table contents and constraint behaviour
- Manual browser testing — verifying the React frontend's behaviour and appearance

8.4 Test Cases and Results

Table 8.1 lists the business-rule test cases carried over from the project's original test log. They were executed on the original implementation, and the same rules are implemented in the Event Service. Table 8.2 lists the microservice-specific tests.

Table 8.1: Business-rule test cases (from the original test log)

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401, login rejected	Pass
2	Student token calls an admin-only route	403 Forbidden	Pass
3	Register for the same event twice	Second attempt returns 409 Conflict	Pass
4	Assign a Maintenance-status room to an event	409, room status rejected	Pass
5	Assign a room smaller than expected participants	409, capacity message with exact numbers	Pass
6	Assign a room already booked for an overlapping time	409, conflict message naming the clashing event	Pass
7	Request a room for a slot starting exactly when another ends	No conflict (back-to-back allowed)	Pass
8	Create a room with negative capacity	400, rejected before reaching the database	Pass
9	Create a room with a duplicate room number + building	409, friendly duplicate message (no raw SQL error)	Pass

#	Test Case	Expected Result	Actual Result
10	Reduce a room's capacity below an upcoming event's needs	409 warning listing the affected event(s)	Pass
11	Delete a room with existing booking history	Room marked Inactive instead of deleted	Pass
12	Delete an event with registrations and a room assignment	Cascades cleanly; zero orphaned rows afterward	Pass
13	Create an occupancy record with a negative headcount	400, rejected	Pass

Table 8.2: Microservice test cases

#	Test Case	Expected Result	Actual Result	Status
14	API Gateway health request	Gateway responds that it is running	Gateway responded with success: true and status running	Pass
15	Gateway routing to User, Admin, and Event Services	Each request reaches its target service	Requests were successfully forwarded to the respective User, Admin, and Event Services	Pass
16	Student registration and login directly on User Service	Account created; JWT returned	Student account was created successfully and a JWT token was returned after login	Pass
17	Admin login directly on Admin Service	JWT returned	Admin login succeeded and a JWT token was returned	Pass
18	Event Service request directly (rooms/events)	Valid response with valid token	Room and event requests returned valid responses with a valid token	Pass
19	Student login through the gateway	JWT returned via gateway	Student login through the API Gateway succeeded and JWT was returned	Pass
20	Admin login through the gateway	JWT returned via gateway	Admin login through the API Gateway succeeded and JWT was returned	Pass

#	Test Case	Expected Result	Actual Result	Status
21	Protected route without token / with student token on admin route	401 / 403	Unauthorized requests returned 401 , and unauthorized role access returned 403	Pass
22	Room assignment with admin token through the gateway	Room assigned; schedule and booking written	Room was successfully assigned and corresponding schedule and booking records were created	Pass
23	Room conflict detection through the gateway	409 with clashing event named	Conflicting room assignment was rejected with 409 Conflict and the clashing event was identified	Pass
24	Student registration for an event	Registration created; duplicate rejected	Registration was created successfully and duplicate registration was rejected	Pass
25	Occupancy update after registration	Effective occupancy, percentage, and status updated	Registered participant count, effective occupancy, percentage, and occupancy status were updated correctly	Pass
26	Cross-service flow: token from User/Admin Service accepted by Event Service	Protected Event Service operation succeeds	JWT issued by the User/Admin Service was accepted by the Event Service and the protected operation succeeded	Pass

One test in the original testing phase uncovered a real defect rather than confirming correct behaviour: pre-filling the Edit Event form's date/time fields initially used the API's raw UTC-encoded timestamp instead of converting it to the browser's local time. This is discussed further in Chapter 10.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration. Each component is started independently, in the following order:

- Start the MySQL server (port 3306) and load the schema and seed data for the three logical databases (user_service_db, admin_service_db, event_service_db).
- Start the User Service (npm install, npm run dev; port 5001).
- Start the Admin Service (npm install, npm run dev; port 5002).
- Start the Event Service (npm install, npm run dev; port 5003).
- Start the API Gateway (npm install, npm run dev; port 4000).
- Start the frontend (npm install, npm run dev; port 5173).
- Open <http://localhost:5173> in a browser and log in with a seeded account.

The services are independent of each other at start-up, but the gateway and frontend require the services to be running before requests can be served. Table 9.1 summarizes the deployment.

Table 9.1: Final local deployment

Component	Technology	Port	Responsibility
Frontend	React.js, Vite	5173	User interface
API Gateway	Node.js, Express.js	4000	Request routing and header forwarding
User Service	Node.js, Express.js	5001	Student accounts and authentication
Admin Service	Node.js, Express.js	5002	Administrator accounts and authentication
Event Service	Node.js, Express.js	5003	Events, rooms, registrations, bookings, occupancy
MySQL	MySQL 8	3306	user_service_db, admin_service_db, event_service_db

9.2 Environment Configuration

Each service keeps its own configuration in a .env file (never committed to version control), covering the database connection for that service's own database (host, port, user, password, database name), the port on which it listens, and, for services that issue or verify tokens, the JWT signing secret and expiry. The API Gateway's configuration contains its port and the addresses of the services. A .env.example template with placeholder values documents the required variables without exposing any real credentials. The signing secret must be identical in every service that issues or verifies tokens.

9.3 Hosting of Frontend and Backend

For this submission, the frontend, the gateway, and the three services run as local Node.js processes and the databases run in a local MySQL instance, which is appropriate for development and in-class demonstration. Vite's development proxy forwards /api calls to the API Gateway, avoiding cross-origin configuration during development. A production deployment was not required for this course submission; Chapter 11 outlines possible future steps.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Table 10.1 lists the main issues encountered and the solutions applied.

Table 10.1: Challenges and solutions

Issue	Solution Applied
Ambiguity between ROOM_SCHEDULES and BOOKINGS: both tables could track “room + time”, risking inconsistency.	ROOM_SCHEDULES was made the single source for every conflict check; BOOKINGS is written in the same transaction purely as an audit trail.
Timezone bug: the Edit Event form was pre-filled by slicing the UTC timestamp string, which is correct only on a UTC machine.	A shared utility (dateUtils.js) converts timestamps with JavaScript local-time getters; verified by simulating the Asia/Kolkata timezone.
MySQL startup failure (“unable to setup unix socket lock file”) caused by a stale mysqld process.	The stale process was terminated, leftover socket and PID files removed, and the service restarted; documented in the README.
JWT compatibility between services: a token issued by one service had to be accepted by another.	Services that issue and verify tokens were configured with the same JWT_SECRET, and token handling was aligned across services.
Admin JWT field mismatch during room assignment.	The token field names used by the Admin Service and read by the Event Service were aligned.
Gateway routing and separate service start-up: requests failed when a route or a service was not running.	Gateway routes were corrected and each service's port was fixed; requests were debugged through the gateway and directly against the services.
Database separation: data formerly in one database had to be divided by owner.	Tables were assigned to user_service_db, admin_service_db, and event_service_db, and cross-database foreign keys were not used.
Synchronizing occupancy information with registrations.	Occupancy was changed to use effective occupancy (actual occupancy plus registered participants) against expected participants.

10.2 Solutions Applied

The solutions are summarized in Table 10.1. In each case the fix was verified by repeating the failing request or workflow after the change, directly against the service and through the gateway where relevant.

10.3 Current Limitations or Known Bugs

- No email verification or password-reset flow currently exists for user accounts.
- Admin accounts can only be created by inserting a row directly into the database — by design, there is no public sign-up path to an admin role.
- Actual occupancy must be entered manually by an admin after an event; there is no automatic check-in mechanism (e.g. QR-code scanning).
- The system runs as local processes; it has not been deployed to a public cloud environment, and the services are not containerized.
- Cross-service consistency relies on the service APIs and shared token conventions; there is no automated integration test suite.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Email-based password reset and account verification.
- A safe, audited admin UI for promoting an existing user to an admin role, rather than requiring direct database access.
- A calendar view of room schedules, so an admin can see a room's bookings visually across a week or month.
- Email or SMS notifications for registration confirmation and event reminders.
- QR-code based check-in at events, so actual occupancy is captured automatically instead of typed in by hand.

11.2 Possible Integrations or Optimizations

The items in Table 11.1 are future enhancements only; none of them has been implemented in the current system.

Table 11.1: Future enhancements

Enhancement	Purpose
Docker / containerization	Consistent, reproducible start-up of the gateway, services, and databases
Service discovery	Avoid fixed service addresses in the gateway configuration
Centralized logging and monitoring	Trace requests across the gateway and services
Automated integration testing (e.g. Jest and Supertest)	Catch regressions automatically, in addition to manual Postman and curl testing
Production deployment and load balancing	Host the system outside a local machine and distribute traffic
API rate limiting	Limit excessive requests at the gateway
Calendar library (e.g. FullCalendar)	Visual room-schedule view
Caching of room-availability queries	Reduce repeated database load during peak registration periods

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace informal, error-prone room coordination on a college campus with a single, database-driven web application. The project began with a centralized backend design — one Express API and one MySQL database — and evolved into a service-oriented backend consisting of an API Gateway, a User Service, an Admin Service, and an Event Service, with separate logical databases for each service. A React frontend for both students and administrators communicates through the gateway. The system was designed, built, and tested incrementally, with each layer verified against a running database.

12.2 What Was Achieved

The finished system enforces JWT-based, role-based authentication; prevents a room from being double-booked through one consistently-applied overlap rule; enforces registration rules (no duplicates, capacity-aware limits); calculates effective occupancy and occupancy status; and gives administrators a live dashboard and management of rooms, events, bookings, registrations, and occupancy. The services are separated by responsibility and data ownership, and the frontend uses one gateway as its common API entry point. The system runs as a local development deployment.

12.3 Skills Learned

Working on this project involved practical experience in relational database design and normalization; writing and enforcing constraints at the database layer; designing REST APIs with consistent response conventions and meaningful HTTP status codes; managing multi-table database transactions; restructuring a monolithic backend into services with an API Gateway; handling JWT authentication across services; building a React frontend with Context-based state management; and systematic testing — including deliberately trying to break the system's own rules to confirm they hold.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vite.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- Axios — Official Documentation, <https://axios-http.com>
- JSON Web Tokens — Introduction, <https://jwt.io/introduction>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core room-conflict overlap query, used consistently across the Event Service:

```
// The single overlap rule used everywhere a room's schedule is checked
// (room availability search, event room-assignment, booking creation):
SELECT rs.room_id, rs.start_time, rs.end_time, e.event_name
FROM room_schedules rs
JOIN events e ON rs.event_id = e.event_id
WHERE rs.status = 'Scheduled'
      AND rs.start_time < ?    -- requested end time
      AND rs.end_time > ?;    -- requested start time
```

Appendix B: Installation and Setup Instructions

Each component is started in its own terminal, in this order.

- Start MySQL (port 3306) and load the schema and seed scripts for user_service_db, admin_service_db, and event_service_db.
- User Service: cd microservices\user-service && npm install && npm run dev (port 5001)
- Admin Service: cd microservices\admin-service && npm install && npm run dev (port 5002)
- Event Service: cd microservices\event-service && npm install && npm run dev (port 5003)
- API Gateway: cd microservices\gateway && npm install && node server.js (port 4000)
- Frontend: cd client && npm install && npm run dev (port 5173)
- Open <http://localhost:5173> in a browser.

Each service requires its own .env file based on its .env.example template (database connection, port, and, where needed, the JWT secret). Real credentials and secrets are not included in this report.

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”.

Table C.1: Seeded accounts

Role	Email	Authenticated By
Admin	admin@college.edu	Admin Service
Student	priyanka.k@college.edu	User Service
Student	geya.reddy@college.edu	User Service
Student (others)	rahul.verma@college.edu, ananya.rao@college.edu, kiran.kumar@college.edu, sneha.patil@college.edu	User Service

Appendix D: API Reference Summary

The endpoints below were carried over from the original API and are organized by the service that now provides them. Full request/response examples are in the accompanying Postman collection.

Table D.1: API reference by service

Service	Resource	Endpoints
User Service (5001)	Auth (students)	POST /register, POST /login, GET /me
Admin Service (5002)	Auth (administrators)	POST /login, GET /me

Service	Resource	Endpoints
Event Service (5003)	Rooms	GET /, GET /:id, GET /available, POST /, PUT /:id, DELETE /:id
Event Service (5003)	Events	GET /, GET /:id, POST /, PUT /:id, DELETE /:id, PUT /:id/assign-room
Event Service (5003)	Registrations	POST /, GET /my, GET /event/:eventId, GET /
Event Service (5003)	Bookings	GET /, POST /, PUT /:id, DELETE /:id
Event Service (5003)	Occupancy	GET /, POST /, PUT /:id
API Gateway (4000)	Health	Gateway health endpoint

Appendix E: Screenshots

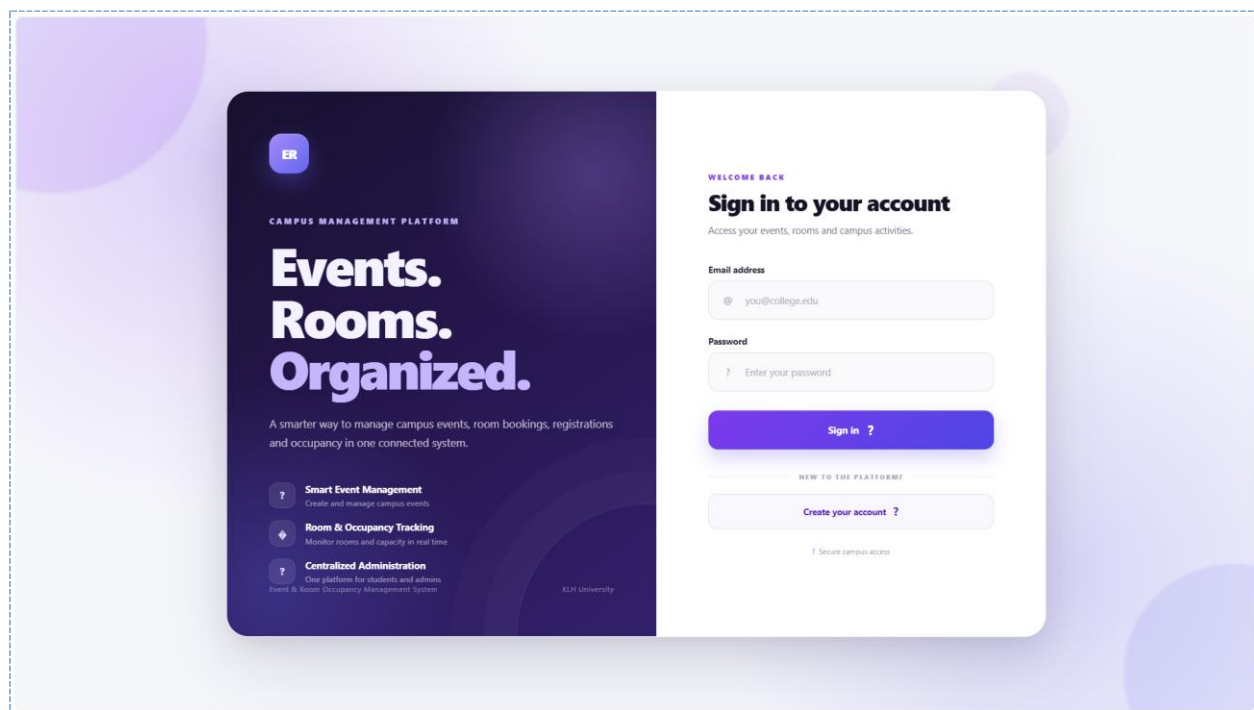


Figure E.1: Login Page

The login page of the React application.

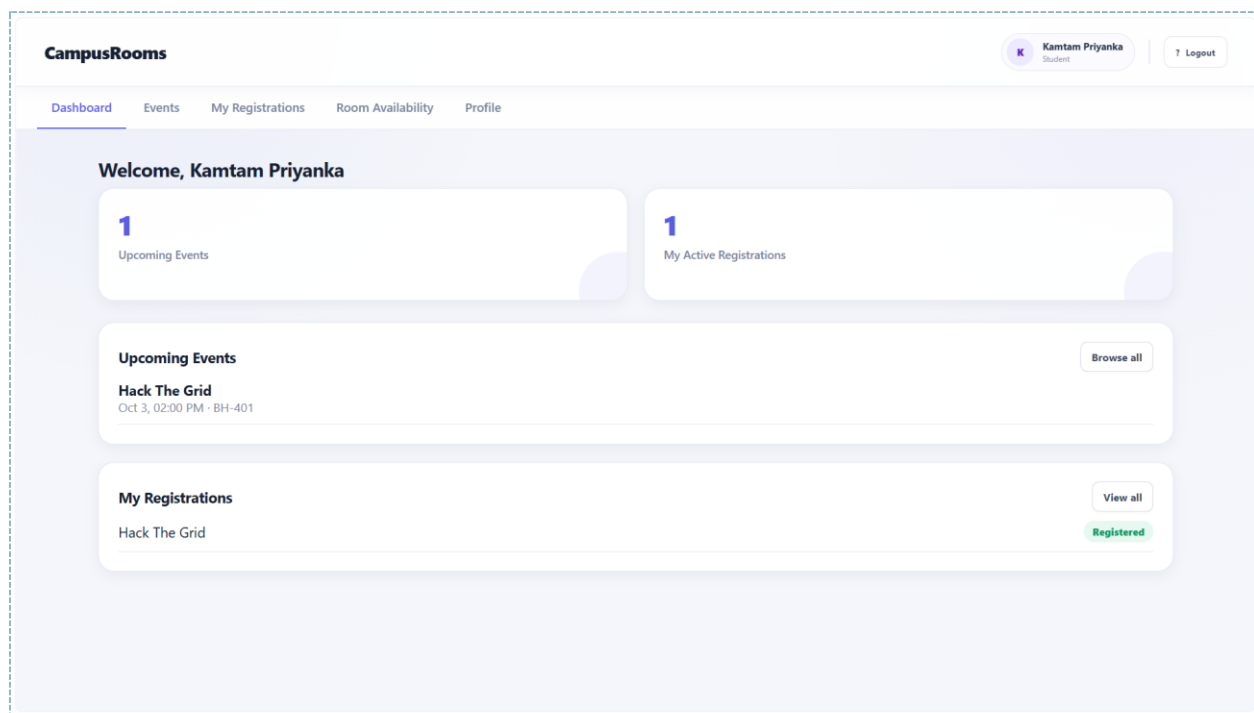


Figure E.2: Student Dashboard

The student home page after login through the gateway.

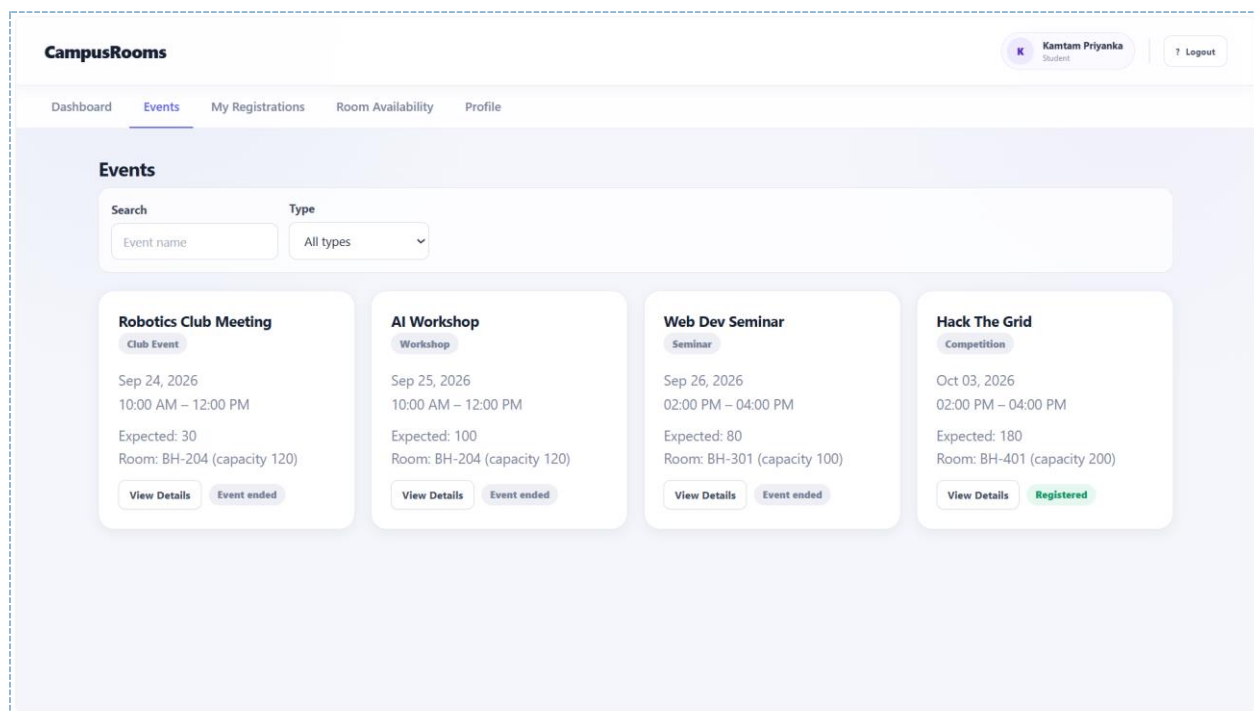


Figure E.3: Student Events Page

Events list with search and type filter; registration is available from an event card.

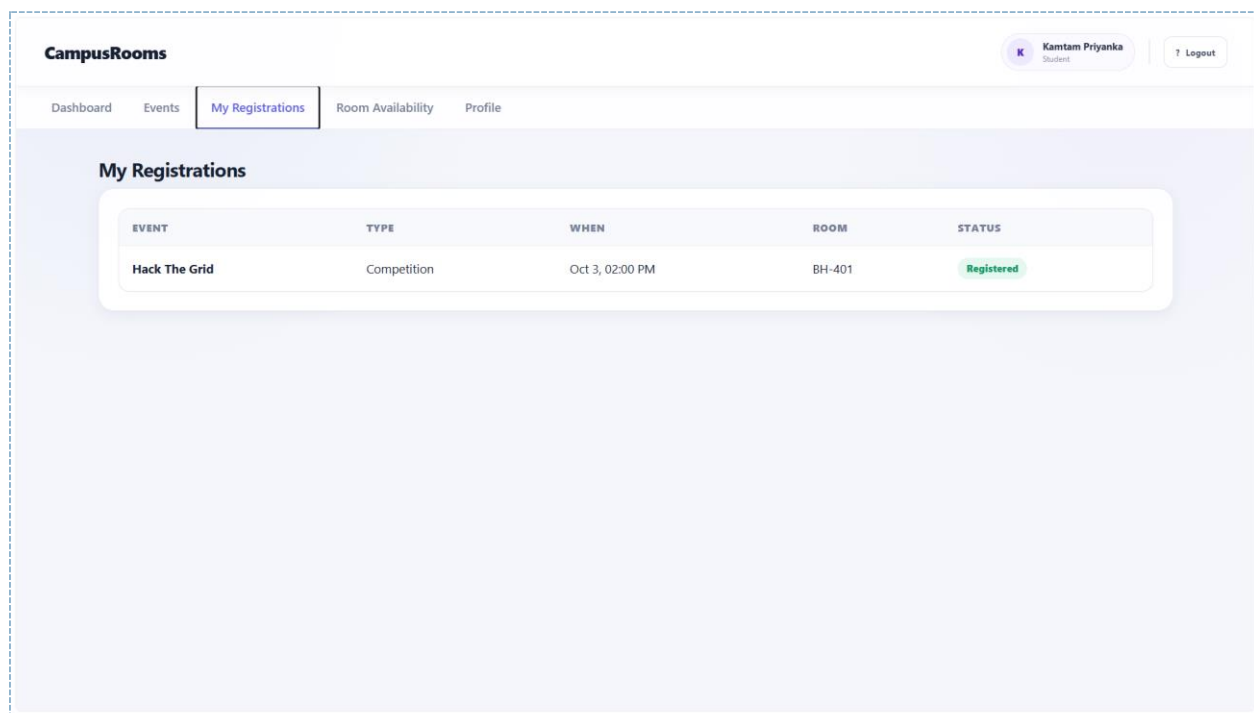


Figure E.4: My Registrations

The logged-in student's registrations and their status.

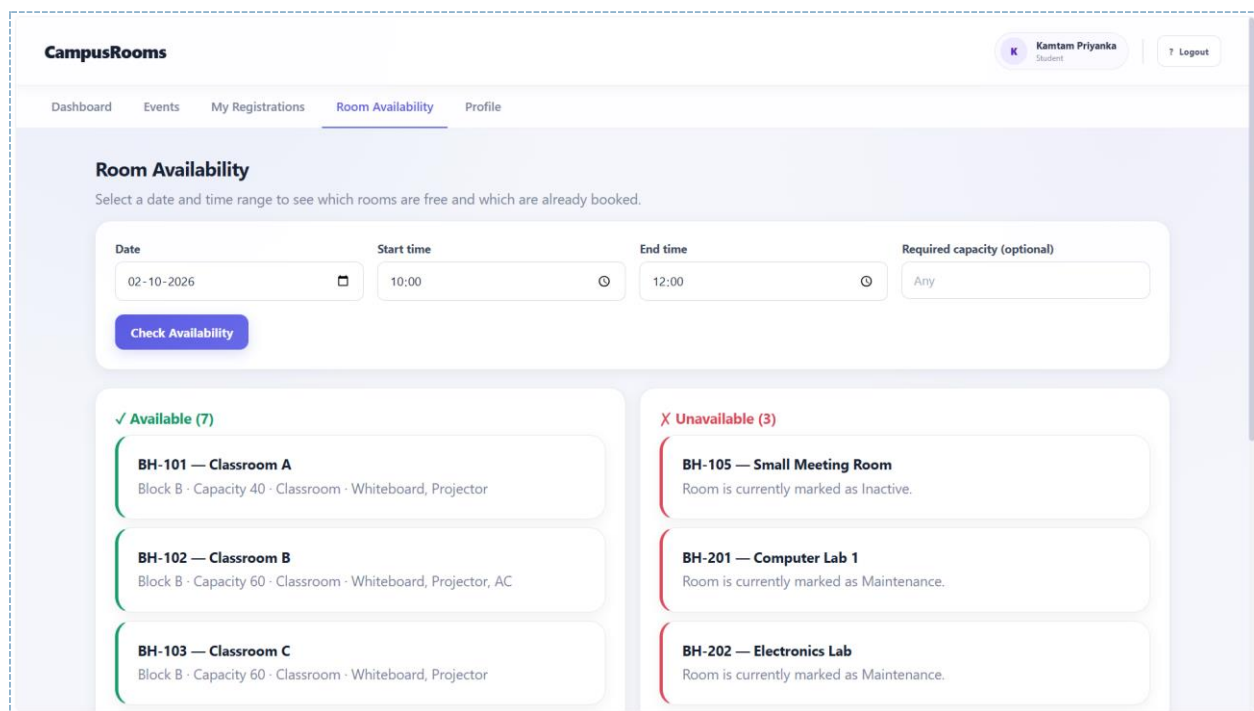


Figure E.5: Room Availability

Available and unavailable rooms for a selected date and time window.

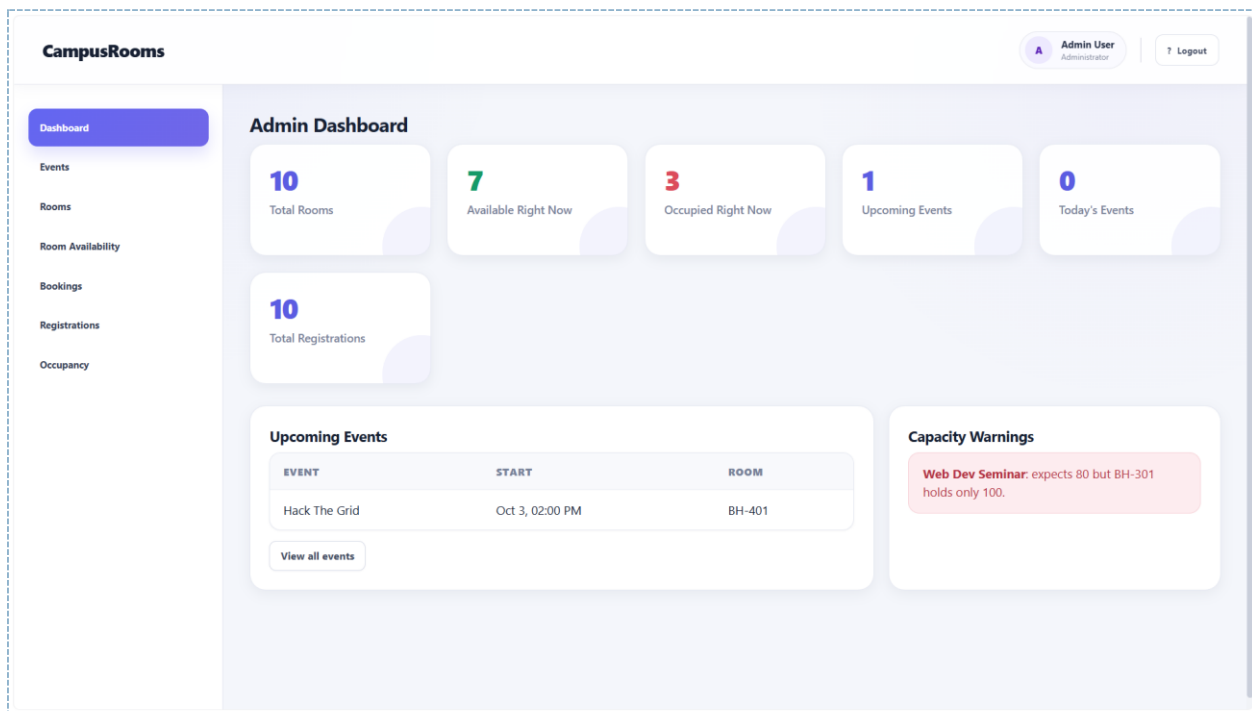


Figure E.6: Admin Dashboard

The admin dashboard after admin login.

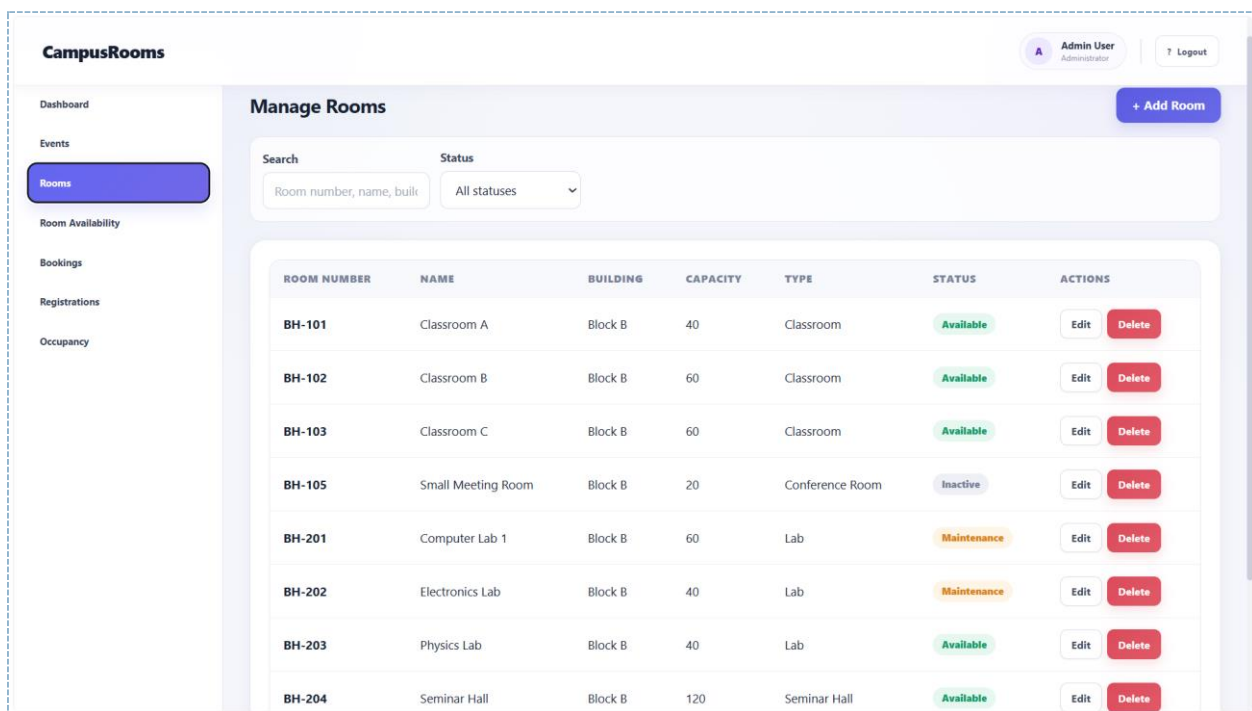


Figure E.7: Manage Rooms

Room list with the Add/Edit Room modal.

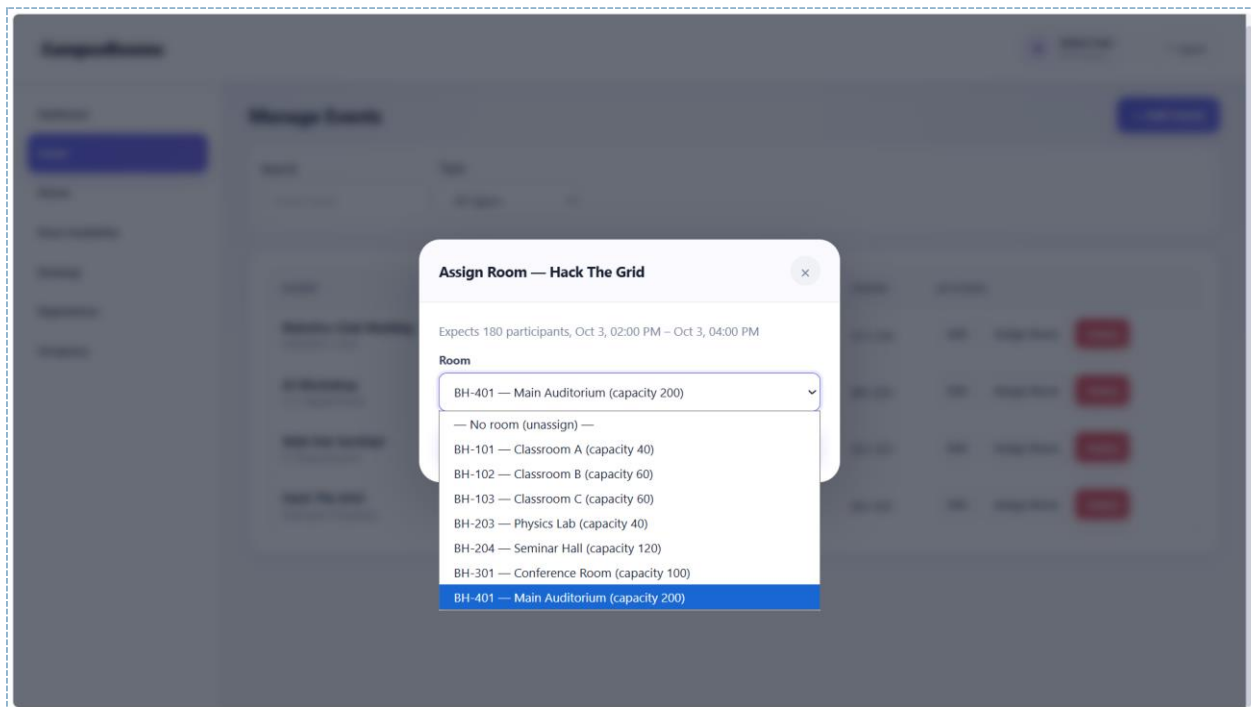


Figure E.8: Manage Events

Event list with the Assign Room modal.

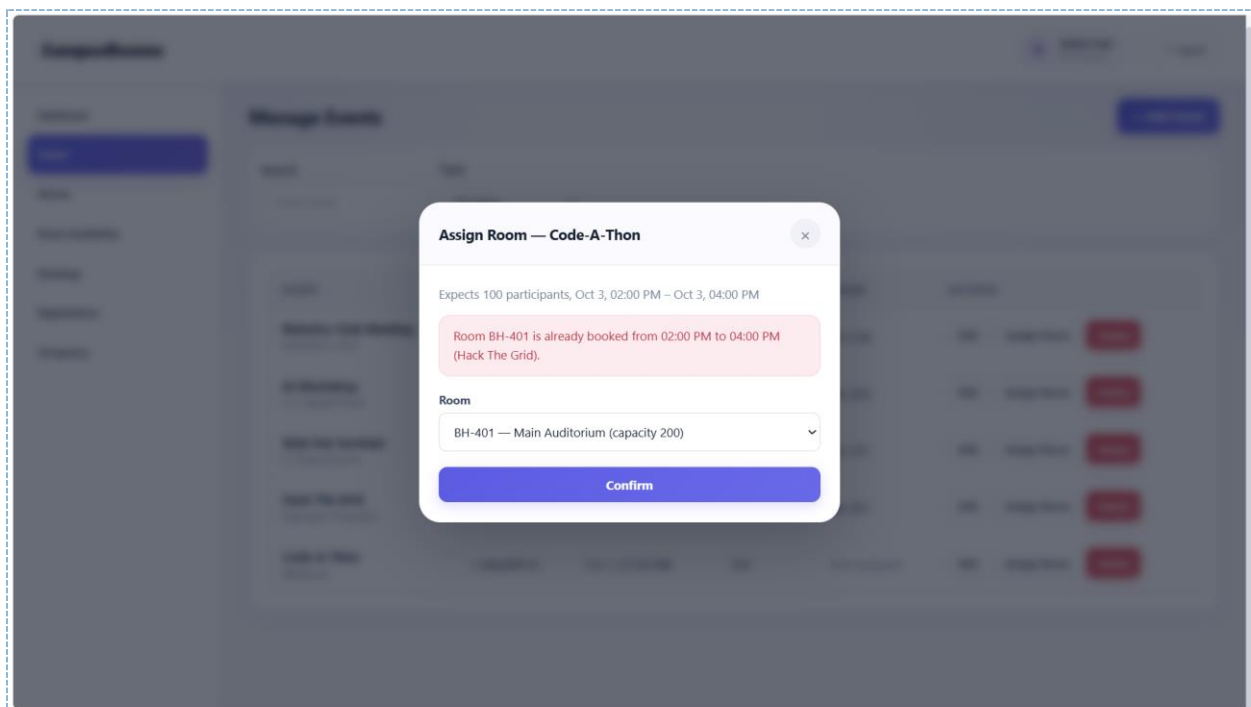


Figure E.9: Room Conflict / Capacity Validation Message

The message shown when a room assignment is rejected for conflict or capacity.

CampusRooms

Admin User

Administrator

Logout

Events

Rooms

Room Availability

Bookings

Registrations

Occupancy

Event

All events

Status

All statuses

STUDENT	EMAIL	EVENT	REGISTERED ON	STATUS
Kamtam Priyanka	kamtampriyanka@klh.edu.in	Hack The Grid	02/10/2026	Registered
Geya Reddy	geya.reddy@college.edu	Hack The Grid	01/10/2026	Registered
Geya Reddy	geya.reddy@college.edu	AI Workshop	29/09/2026	Registered
Priyanka Kamtam	priyanka.k@college.edu	AI Workshop	29/09/2026	Registered
Rahul Verma	rahul.verma@college.edu	AI Workshop	29/09/2026	Registered
Ananya Rao	ananya.rao@college.edu	AI Workshop	29/09/2026	Registered
Sneha Patil	sneha.patil@college.edu	Web Dev Seminar	29/09/2026	Registered
Rahul Verma	rahul.verma@college.edu	Web Dev Seminar	29/09/2026	Cancelled
Ananya Rao	ananya.rao@college.edu	Robotics Club Meeting	29/09/2026	Registered
Kiran Kumar	kiran.kumar@college.edu	Robotics Club Meeting	29/09/2026	Attended

Figure E.10: Registrations Management

All student registrations with event and status filters.

CampusRooms

Admin User

Administrator

Logout

Dashboard

Events

Rooms

Room Availability

Bookings

Registrations

Occupancy

Occupancy Records

+ Add Record

EVENT	ROOM	DATE	EXPECTED	REGISTERED	ACTUAL	OCCUPANCY	
Hack The Grid	BH-401	03/10/2026	180	2	120	68% — High	Update
Web Dev Seminar	BH-301	26/09/2026	80	1	80	101% — Over Capacity	Update
AI Workshop	BH-204	25/09/2026	100	4	70	74% — High	Update
Robotics Club Meeting	BH-101	24/09/2026	30	1	18	63% — High	Update

Figure E.11: Occupancy Management

Occupancy records showing effective occupancy, percentage, and status.

Figure E.12: API Gateway Health Response

Response of the gateway health endpoint on port 4000.

```
PS C:\WINDOWS\System32> cd "D:\Project\Event-Room-Management\microservices\gateway"
PS D:\Project\Event-Room-Management\microservices\gateway> node server.js
API Gateway running on http://localhost:4000
```

Figure E.13: Postman — User Service

```
PS C:\WINDOWS\System32> cd "D:\Project\Event-Room-Management\microservices\user-service"
PS D:\Project\Event-Room-Management\microservices\user-service> npm run dev

> user-service@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (6) from .env
◇ injected env (0) from .env
User Service running on http://localhost:5001
```

Request to the User Service.

Figure E.14: Postman — Admin Service

```
PS C:\WINDOWS\System32> cd "D:\Project\Event-Room-Management\microservices\admin-service"
PS D:\Project\Event-Room-Management\microservices\admin-service> npm run dev

> admin-service@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (6) from .env
◇ injected env (0) from .env
Admin Service running on http://localhost:5002
```

Request to the Admin Service.

Figure E.15: Postman — Event Service

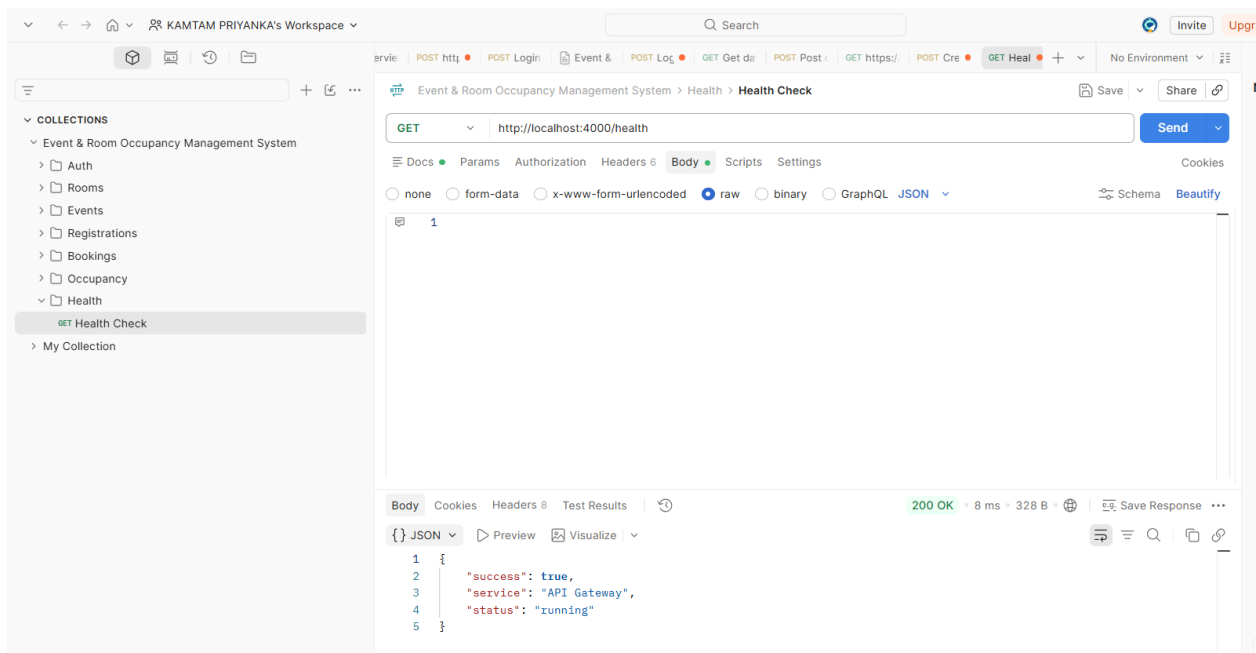
```
PS C:\WINDOWS\System32> cd "D:\Project\Event-Room-Management\microservices\event-service"
PS D:\Project\Event-Room-Management\microservices\event-service> npm run dev

> event-service@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (6) from .env
◇ injected env (0) from .env
Event Service running on http://localhost:5003
```

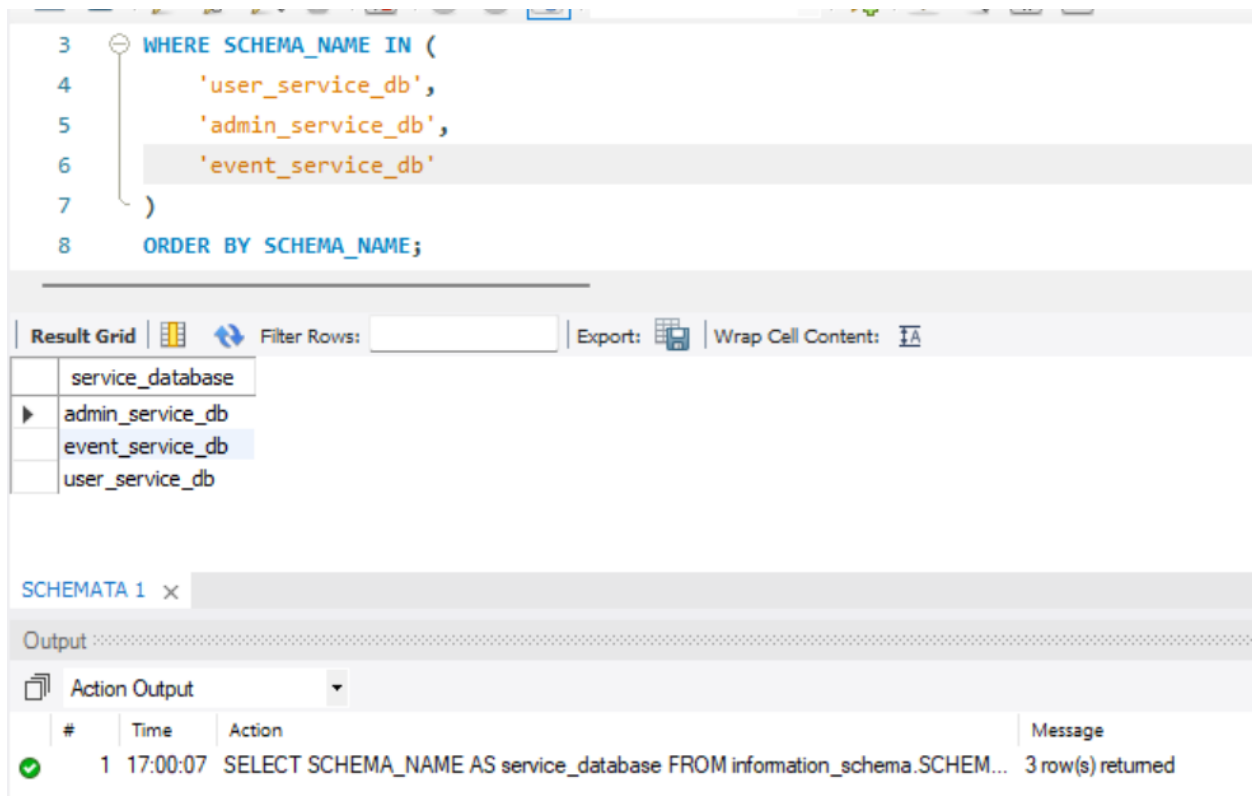
Request to the Event Service.

Figure E.16: Postman — Gateway Request



Request sent through the API Gateway.

Figure E.17: MySQL Service Databases



user_service_db, admin_service_db, and event_service_db in MySQL Workbench.

The final microservice architecture diagram and the final ER / database architecture diagram are included in the report as Figure 4.1 and Figure 5.1.